

**Образовательная автономная некоммерческая организация
высшего образования**

«МОСКОВСКИЙ ТЕХНОЛОГИЧЕСКИЙ ИНСТИТУТ»

Факультет информационных технологий
Направление подготовки 09.04.03 «**Прикладная информатика**»

Отчёт по лабораторному практикуму № 2

**«Классификация и сравнение
систем управления версиями»
по дисциплине**

«Системы управления версиями»

Выполнил студент гр.
ОВМП-103ивс

Осипов Д. С.

Принял _____
(Должность Руководителя)

Дмитриев Д. В.
(ФИО Руководителя)

Москва, 2026 г.

1. Цель работы

Освоить классификацию систем управления версиями (локальные, централизованные и распределённые), получить практический опыт работы с централизованной (Apache Subversion, SVN) и распределённой (Git) системами и провести их сравнительный анализ по критериям удобства, скорости и надёжности.

Среда выполнения: macOS 26.6 (Apple Silicon), оболочка zsh; Git 2.54.0 (Apple Git-156), Subversion 1.14.5 (установлен через Homebrew: `brew install subversion`). Работа выполнялась в каталоге `~/СУВ_ЛП2`.

2. Теоретическая таблица сравнения архитектур (Этап 1)

Таблица 1 — Сравнение архитектур централизованной и распределённой СУБ

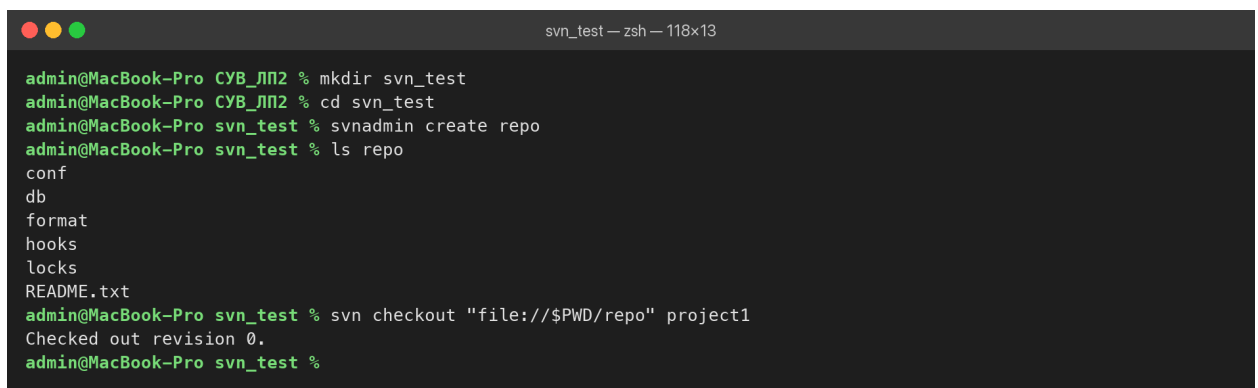
Критерий	Централизованная (SVN)	Распределённая (Git)
Где хранится каноническая история	На центральном сервере (в репозитории, созданном <code>svnadmin create</code>). Рабочая копия хранит только файлы текущей ревизии и служебные метаданные <code>.svn</code>	В каждом клоне хранится полная история доступных веток (каталог <code>.git</code>). «Каноническим» по соглашению команды считается удалённый репозиторий (<code>origin</code> , GitHub/GitLab)
Работа без сети	Можно редактировать файлы, выполнять <code>svn status</code> и <code>svn diff</code> . <code>Commit</code> , <code>update</code> , <code>log</code> по серверной истории, создание веток и тегов требуют доступа к серверу	Доступны коммиты, просмотр истории, <code>diff</code> , создание веток, слияния. Сеть нужна только для обмена изменениями (<code>push</code> , <code>pull</code> , <code>fetch</code>)
Создание веток	Серверная операция <code>svn</code> <code>сору</code> в каталог <code>branches/</code> — дешёвая копия, но требует связи с репозиторием и фиксируется как новая ревизия	Локальная операция: ветка — лёгкий указатель на коммит, создаётся мгновенно (<code>git switch -c</code>)
Хранение крупных бинарных файлов	Удобнее: рабочая копия не содержит всю историю, поддерживаются блокировки файлов (<code>svn lock</code>) для нессливаемых ассетов	Обычный Git хранит все версии в каждом клоне, репозиторий быстро растёт. Применяют Git LFS, <code>shallow/partial clone</code> или специализированные системы (Perforce)
Типичные сценарии применения	Legacy-проекты, корпоративные репозитории с правами по каталогам, проекты с крупными бинарными файлами и блокировками (игровые и CAD-ассеты)	Разработка исходного кода любого масштаба, <code>open source</code> , распределённые команды, DevOps и CI/CD, встраиваемое ПО (Linux, Zephyr)

3. Пошаговое выполнение заданий (Этапы 2–3)

Ниже приведены изображения окна терминала с реальным выводом команд, выполненных в ходе работы (полный журнал команд с выводом и временем выполнения сохранён в файле `terminal_log.json`).

3.1. Работа с SVN (Этап 2)

Шаг 1. Создана папка `svn_test`, в ней командой `svnadmin create` создан репозиторий `repo` (каталоги `conf`, `db`, `hooks`, `locks` — серверная часть SVN). Затем получена первая рабочая копия `project1` командой `svn checkout` по URL `file:///.../repo` (рисунок 1). Поскольку репозиторий пуст, получена ревизия 0.



```
admin@MacBook-Pro CYB_ЛП2 % mkdir svn_test
admin@MacBook-Pro CYB_ЛП2 % cd svn_test
admin@MacBook-Pro svn_test % svnadmin create repo
admin@MacBook-Pro svn_test % ls repo
conf
db
format
hooks
locks
README.txt
admin@MacBook-Pro svn_test % svn checkout "file://$PWD/repo" project1
Checked out revision 0.
admin@MacBook-Pro svn_test %
```

Рисунок 1 — Создание SVN-репозитория и первой рабочей копии

Шаг 2. В рабочей копии создан файл `readme.txt` с текстом «Hello SVN», файл поставлен под контроль версий (`svn add` — статус A) и зафиксирован командой `svn commit -m "Initial commit"`. Сервер присвоил изменению ревизию 1 (рисунок 2).



```
admin@MacBook-Pro svn_test % cd project1
admin@MacBook-Pro project1 % echo "Hello SVN" > readme.txt
admin@MacBook-Pro project1 % svn add readme.txt
A      readme.txt
admin@MacBook-Pro project1 % svn status
A      readme.txt
admin@MacBook-Pro project1 % svn commit -m "Initial commit"
Adding      readme.txt
Transmitting file data .done
Committing transaction...
Committed revision 1.
admin@MacBook-Pro project1 %
```

Рисунок 2 — Добавление `readme.txt` и первый коммит в SVN

Шаг 3. В файл добавлена вторая строка; `svn diff` показывает разницу между рабочей копией и ревизией 1. Изменение зафиксировано как ревизия 2

(рисунок 3).

```
project1 — zsh — 118x15

admin@MacBook-Pro project1 % echo "Вторая строка: добавлено описание проекта" >> readme.txt
admin@MacBook-Pro project1 % svn diff
Index: readme.txt
=====
--- readme.txt[(revision 1)]
+++ readme.txt[(working copy)]
@@ -1,2 @@
 Hello SVN
+Вторая строка: добавлено описание проекта
admin@MacBook-Pro project1 % svn commit -m "Добавлена вторая строка в readme.txt"
Sending          readme.txt
Transmitting file data .done
Committing transaction...
Committed revision 2.
admin@MacBook-Pro project1 %
```

Рисунок 3 — Изменение файла, svn diff и второй коммит

Шаг 4. Просмотр истории командой `svn log` в первой папке. Вывод пустой — только строка-разделитель, а `svn info` показывает, что сама папка `project1` по-прежнему находится на ревизии 0 (рисунок 4).

```
project1 — zsh — 118x5

admin@MacBook-Pro project1 % svn log
-----
admin@MacBook-Pro project1 % svn info --show-item revision
0
admin@MacBook-Pro project1 %
```

Рисунок 4 — `svn log` в первой рабочей копии сразу после коммитов

Шаг 5. Командой `svn checkout` получена вторая рабочая копия `project2`. В ней `svn log` сразу выводит обе ревизии — `r2` и `r1` с автором, датой и сообщением (рисунок 5).

```
project2 — zsh — 118x16

admin@MacBook-Pro project1 % cd ..
admin@MacBook-Pro svn_test % svn checkout "file://$PWD/repo" project2
A    project2/readme.txt
Checked out revision 2.
admin@MacBook-Pro svn_test % cd project2
admin@MacBook-Pro project2 % svn log
-----
r2 | admin | 2026-10-05 19:29:11 +0300 (пн, 05 окт. 2026) | 1 line
Добавлена вторая строка в readme.txt
-----
r1 | admin | 2026-10-05 19:29:10 +0300 (пн, 05 окт. 2026) | 1 line
Initial commit
-----
admin@MacBook-Pro project2 %
```

Рисунок 5 — Вторая рабочая копия: `svn log` показывает ревизии `r1` и `r2`

Ответ на вопрос «Чего не хватает в первой папке проекта». В первой рабочей копии не хватает актуального состояния каталога: после `svn commit`

SVN обновляет номер ревизии только у отправленного файла, а сама папка остаётся на прежней ревизии (смешанная рабочая копия, mixed revisions). Команда `svn log` без аргументов показывает историю только до ревизии рабочей копии, поэтому `r1` и `r2` «не видны». Вывод: в централизованной СУВ локальная копия не содержит историю — она хранится на сервере, и чтобы увидеть её, нужно синхронизироваться командой `svn update`. После `svn update` папка перешла на ревизию 2, и `svn log` вывел полную историю (рисунок 6).

A terminal window titled 'project1 — zsh — 118x15'. The user 'admin@MacBook-Pro' is in the 'project2' directory. They run 'cd ../project1', then 'svn update'. The output shows 'Updating '.'' and 'At revision 2.'. Then they run 'svn log', which shows two revisions: 'r2' (revision 2) and 'r1' (revision 1). The commit message for r2 is 'Добавлена вторая строка в readme.txt'. The commit message for r1 is 'Initial commit'.

```
admin@MacBook-Pro project2 % cd ../project1
admin@MacBook-Pro project1 % svn update
Updating '.'
At revision 2.
admin@MacBook-Pro project1 % svn log
-----
r2 | admin | 2026-10-05 19:29:11 +0300 (пн, 05 окт. 2026) | 1 line
Добавлена вторая строка в readme.txt
-----
r1 | admin | 2026-10-05 19:29:10 +0300 (пн, 05 окт. 2026) | 1 line
Initial commit
-----
admin@MacBook-Pro project1 %
```

Рисунок 6 — После `svn update` первая рабочая копия видит всю историю

3.2. Работа с Git (Этап 3)

Шаг 1. Создана папка `git_test`, в ней командой `git init --bare` создан «голый» репозиторий `repo.git` — аналог сервера (без рабочей директории). Репозиторий клонирован в папку проекта `project1`; Git предупреждает, что клонирован пустой репозиторий (рисунок 7).

A terminal window titled 'git_test — zsh — 118x10'. The user 'admin@MacBook-Pro' is in the 'project1' directory. They run 'cd ~/СУВ_ЛП2', then 'mkdir git_test', then 'cd git_test'. They run 'git init --bare repo.git', which outputs 'Initialized empty Git repository in /Users/admin/СУВ_ЛП2/git_test/repo.git/'. Then they run 'git clone repo.git project1', which outputs 'Cloning into 'project1'...' and a warning: 'warning: You appear to have cloned an empty repository. done.'. The prompt returns to 'admin@MacBook-Pro git_test %'.

```
admin@MacBook-Pro project1 % cd ~/СУВ_ЛП2
admin@MacBook-Pro СУВ_ЛП2 % mkdir git_test
admin@MacBook-Pro СУВ_ЛП2 % cd git_test
admin@MacBook-Pro git_test % git init --bare repo.git
Initialized empty Git repository in /Users/admin/СУВ_ЛП2/git_test/repo.git/
admin@MacBook-Pro git_test % git clone repo.git project1
Cloning into 'project1'...
warning: You appear to have cloned an empty repository.
done.
admin@MacBook-Pro git_test %
```

Рисунок 7 — Создание bare-репозитория и клонирование в папку проекта

Шаг 2. Указаны имя и e-mail автора для репозитория, создан `readme.txt` с текстом «Hello Git», выполнены `git add .` (файл попал в индекс — статус A) и `git commit -m "Initial commit"` (рисунок 8). Коммит создан локально, без

обращения к `геро.git`.

```
project1 — zsh — 118x12

admin@MacBook-Pro git_test % cd project1
admin@MacBook-Pro project1 % git config user.name "Daniil Osipov"
admin@MacBook-Pro project1 % git config user.email "borodachkiller055@gmail.com"
admin@MacBook-Pro project1 % echo "Hello Git" > readme.txt
admin@MacBook-Pro project1 % git add .
admin@MacBook-Pro project1 % git status --short
A  readme.txt
admin@MacBook-Pro project1 % git commit -m "Initial commit"
[main (root-commit) 0b7b6d9] Initial commit
1 file changed, 1 insertion(+)
create mode 100644 readme.txt
admin@MacBook-Pro project1 %
```

Рисунок 8 — Добавление `readme.txt` в индекс и первый коммит в Git

Шаг 3. Добавлена вторая строка, `git diff` показывает изменение, оно зафиксировано вторым коммитом (рисунок 9).

```
project1 — zsh — 118x13

admin@MacBook-Pro project1 % echo "Second line: project description added" >> readme.txt
admin@MacBook-Pro project1 % git diff
diff --git a/readme.txt b/readme.txt
index 9f4d96d..d32b2e0 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,1,2 @@
 Hello Git
+Second line: project description added
admin@MacBook-Pro project1 % git commit -am "Add second line to readme.txt"
[main e91349d] Add second line to readme.txt
1 file changed, 1 insertion(+)
admin@MacBook-Pro project1 %
```

Рисунок 9 — Изменение файла, `git diff` и второй коммит

Шаг 4. Просмотр истории: `git log` выводит оба коммита с хешами, автором и датой; `git log --oneline --graph` — компактный граф (рисунок 10).

```
project1 — zsh — 118x16

admin@MacBook-Pro project1 % git log
commit e91349dbd44a6112417cd4cf3b32bbf1ff0bfad0
Author: Daniil Osipov <borodachkiller055@gmail.com>
Date:   Mon Oct 5 19:29:14 2026 +0300

    Add second line to readme.txt

commit 0b7b6d9c439b4c17444ea4599af768833ced4193
Author: Daniil Osipov <borodachkiller055@gmail.com>
Date:   Mon Oct 5 19:29:14 2026 +0300

    Initial commit
admin@MacBook-Pro project1 % git log --oneline --graph
* e91349d Add second line to readme.txt
* 0b7b6d9 Initial commit
admin@MacBook-Pro project1 %
```

Рисунок 10 — История коммитов в первом клоне

Шаг 5. Чтобы изменения стали доступны другим клонам, локальная история отправлена в `геро.git` командой `git push` (рисунок 11) — в Git

фиксация (commit) и публикация (push) разделены.



```
project1 — zsh — 118x4
admin@MacBook-Pro project1 % git push origin HEAD
To /Users/admin/CYB_ЛП2/git_test/repo.git
* [new branch]      HEAD -> main
admin@MacBook-Pro project1 %
```

Рисунок 11 — Публикация коммитов в удалённый репозиторий

Шаг 6. Репозиторий клонирован во вторую папку project2; git log в ней показывает ту же историю из двух коммитов, что и в первом клоне (рисунок 12). В отличие от SVN, в первой папке ничего «не пропадает»: история хранится локально в каждом клоне, и git log в обеих папках выводит одинаковый результат без дополнительной синхронизации.

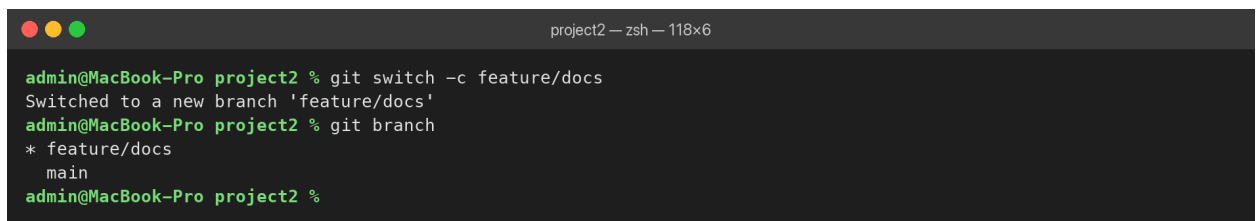


```
project2 — zsh — 118x9
admin@MacBook-Pro project1 % cd ..
admin@MacBook-Pro git_test % git clone repo.git project2
Cloning into 'project2'...
done.
admin@MacBook-Pro git_test % cd project2
admin@MacBook-Pro project2 % git log --oneline
e91349d Add second line to readme.txt
0b7b6d9 Initial commit
admin@MacBook-Pro project2 %
```

Рисунок 12 — Второй клон содержит полную историю проекта

3.3. Дополнительные проверки для сравнительного анализа

Для заполнения итоговой таблицы дополнительно проверены ветвление и работа без доступа к репозиторию. В Git ветка создаётся локально и мгновенно (рисунок 13). В SVN ветка — это серверная копия: потребовалось создать каталог branches и выполнить svn сору, каждая операция стала новой ревизией репозитория (рисунок 14).



```
project2 — zsh — 118x6
admin@MacBook-Pro project2 % git switch -c feature/docs
Switched to a new branch 'feature/docs'
admin@MacBook-Pro project2 % git branch
* feature/docs
  main
admin@MacBook-Pro project2 %
```

Рисунок 13 — Создание ветки в Git

```
svn_test — zsh — 118x8

admin@MacBook-Pro project2 % cd ~/CVB_ЛП2/svn_test
admin@MacBook-Pro svn_test % svn mkdir "file://$PWD/repo/branches" -m "Создан каталог branches"
Committing transaction...
Committed revision 3.
admin@MacBook-Pro svn_test % svn copy "file://$PWD/repo/readme.txt" "file://$PWD/repo/branches/readme-feature.txt" -m
Committing transaction...
Committed revision 4.
admin@MacBook-Pro svn_test %
```

Рисунок 14 — Создание «ветки» в SVN через svn copy

Недоступность сервера имитировалась переименованием каталога репозитория. SVN не смог выполнить svn update и вернул ошибки E170013/E180001 (рисунок 15), а Git при недоступном repo.git успешно создал новый коммит локально (рисунок 16) — его можно будет отправить командой git push после восстановления связи.

```
svn_test — zsh — 118x6

admin@MacBook-Pro svn_test % mv repo repo_offline && cd project1 && svn update; cd .. && mv repo_offline repo
Updating '.':
svn: E170013: Unable to connect to a repository at URL 'file:///Users/admin/%D0%A1%D0%A3%D0%92_%D0%9B%D0%9F2/s
vn_test/repo'
svn: E180001: Unable to open repository 'file:///Users/admin/%D0%A1%D0%A3%D0%92_%D0%9B%D0%9F2/svn_test/repo'
admin@MacBook-Pro svn_test %
```

Рисунок 15 — SVN: операция с недоступным репозиторием завершается ошибкой

```
project1 — zsh — 118x4

admin@MacBook-Pro svn_test % cd ~/CVB_ЛП2/git_test/project1
admin@MacBook-Pro project1 % mv ../repo.git ../repo_offline.git && echo "offline" >> readme.txt && git commit -qam "0
7f55003 Offline commit
admin@MacBook-Pro project1 %
```

Рисунок 16 — Git: коммит выполняется без доступа к удалённому репозиторию

4. Итоговая сравнительная таблица (Этап 4)

Таблица 2 — Итоговое сравнение SVN и Git по результатам практики

Критерий	SVN (централизованная)	Git (распределённая)	Ваши впечатления
Сложность установки	Не входит в macOS; установлен через Homebrew (brew install subversion) вместе с зависимостями	Уже установлен вместе с Xcode Command Line Tools	Git доступен «из коробки»; SVN потребовал отдельной установки, но тоже ставится одной командой
Скорость операций (commit, log)	svn commit $\approx 0,9$ с, svn checkout/update $\approx 0,7\text{--}1,0$ с, svn log $\approx 0,03$ с	git commit $\approx 0,04$ с, git log $\approx 0,03$ с, git push $\approx 0,1$ с	На маленьком проекте разница незаметна для человека, но Git-коммит в ~ 20 раз быстрее, так как выполняется локально
Удобство создания веток	Ветка — копия каталога на сервере (svn copy), требует продуманной структуры trunk/branches/tags и создаёт новую ревизию	Одна команда git switch -с, ветка создаётся мгновенно и локально	В Git ветки «бесплатные», поэтому ими пользуешься постоянно; в SVN ветвление ощущается как отдельная административная операция
Работа без интернета	Без репозитория нельзя коммитить, обновляться и смотреть серверную историю (получили ошибку E170013)	Коммиты, история, ветки и diff работают офлайн; сеть нужна только для push/pull	Git позволяет спокойно работать в дороге и при сбоях сервера
Понятность сообщений об ошибках	Сообщения с кодами (E170013, E180001) и URL; суть понятна, но URL с кириллицей выводится в процентной кодировке	Сообщения подробные, часто с подсказкой следующей команды (например, при клонировании пустого репозитория)	Сообщения Git дружелюбнее для новичка; коды SVN удобны для поиска в документации

5. Обоснование выбора СУВ для заданных сценариев

Сценарий 1 — ядро ОС, 100 разработчиков в разных часовых поясах.

Рекомендуется распределённая СУВ Git. Причины: (1) каждый разработчик работает автономно и коммитит локально независимо от доступности сервера и разницы во времени; (2) лёгкие ветки и слияния позволяют вести сотни параллельных изменений и проверять их через review; (3) полная история в каждом клоне повышает отказоустойчивость, а цепочка доверенных мейнтейнеров (как в проекте Linux, для которого Git и создавался) масштабирует процесс интеграции.

Сценарий 2 — мобильная игра, код на C#, 50 ГБ текстур и 3D-моделей, команда 10 человек.

Рекомендуется Git для исходного кода в сочетании с Git LFS для ассетов и включёнными блокировками файлов LFS (`git lfs lock`) для несливаемых бинарных файлов. Для 10 человек это проще и дешевле в сопровождении, чем отдельный сервер Perforce, при этом сохраняется привычная экосистема (pull request, CI/CD). SVN оправдан, если команда уже использует его или художникам важна простая централизованная модель с блокировками. Если объём ассетов будет расти до сотен гигабайт и они часто меняются, стоит рассмотреть Perforce Helix Core — стандарт игровой индустрии для крупных бинарных данных.

6. Выводы

В ходе работы освоена классификация СУВ и на практике выполнены одинаковые сценарии в централизованной (SVN) и распределённой (Git) системах. Эксперимент наглядно показал ключевое архитектурное различие: в SVN история хранится на сервере, поэтому первая рабочая копия после коммитов «не видела» историю до выполнения `svn update`, а при недоступности репозитория работа остановилась. В Git каждый клон содержит полную историю, коммиты и ветки создаются локально и мгновенно, а сервер нужен только для обмена изменениями.

Для личного использования удобнее Git: он уже установлен, работает офлайн, ветки создаются одной командой, а платформы GitHub/GitLab дают бесплатный удалённый репозиторий. Другую систему я бы выбрал в проекте с большим объёмом несливаемых бинарных файлов и потребностью в блокировках и правах доступа по каталогам — здесь уместны SVN или Perforce, — а также при поддержке legacy-проекта, где миграция на Git не окупается.

7. Список использованных источников

1. Дмитриев Д. В. Классификация и сравнение систем управления версиями: презентация лекции. — М.: МТИ, 2026.
2. Сергеев О. В. Теоретический материал по курсу «Системы управления версиями». — М.: МТИ, 2025.
3. Чакон С., Штрауб Б. Pro Git. — 2-е изд. — URL: <https://git-scm.com/book/ru/v2> (дата обращения: 05.10.2026).
4. Git Documentation. — URL: <https://git-scm.com/docs> (дата обращения: 05.10.2026).
5. Version Control with Subversion (SVN Book). — URL: <https://svnbook.red-bean.com> (дата обращения: 05.10.2026).
6. Apache Subversion. — URL: <https://subversion.apache.org> (дата обращения: 05.10.2026).